Predicting Sprint Outcomes from Reconstructed Jira State: Count Baselines and a Leakage Audit

Hani AbuSharkh
Higher Colleges of Technology
Al Ain, UAE
habusharkh@hct.ac.ae

Maryam Hanif
United Arab Emirates University
Al Ain, UAE
700052696@uaeu.ac.ae

Rafat Damseh
United Arab Emirates University
Al Ain, UAE
rdamseh@uaeu.ac.ae

***Abstract*—A present-day Jira export shows how a sprint ended, not what the team knew when it began, so sprint-risk models trained on it can learn from hindsight. We replay the change log of eight TAWOS projects to each sprint's start (2,602 sprints, 1,691 with committed work) and log where missing history forces a static fallback. We then test planning-time and in-sprint classifiers for late closure and spillover against learning-free baselines. Gradient boosting (GB) ranks late closure at a pooled ROC-AUC of 0.751 on one time-ordered split and 0.696 $\pm$ 0.054 over five. Much of this separates projects. Within a project GB averages 0.634 (95% interval 0.570 to 0.695), and the project's own recent late rate ranks its next sprints at chance (0.510). The remaining signal sits in short administrative lags: 40% of late sprints close one to three days late, and with a seven-day threshold the within-project AUC falls to 0.502. Transfer to ten MariaDB and MongoDB projects is weak. Spillover, meaning any committed issue unfinished at the planned end, is close to mechanical. Only 32% of commitments are resolved by the planned end, the number of committed issues ranks spillover at 0.881, and at mid-sprint the number still open (0.945) cannot be distinguished from the best model (0.957). We offer the results as an audit protocol for sprint analytics.**

***Index Terms*—Scrum, sprint analytics, Jira, machine learning, data leakage, baselines**

I. Introduction

Jira records two sprint outcomes that need no human judgement. A sprint spills over when an issue committed at its start is still unresolved at the planned end, and it closes late when its completion is stamped more than a day after that end. We ask whether either outcome can be ranked from what was known on the first day of the sprint, and whether progress during the sprint adds anything.

A present-day export cannot answer this directly. Issues move between sprints, estimates are revised, and several sprints are often open at once, so the current issue-to-sprint link mixes work committed at planning with work added later. We therefore replay the change log of the TAWOS dataset of agile open-source projects [1] back to each sprint's start and record where a missing history forces a static fallback. A second leak concerns history features. In 71% of our sprints the previous sprint had not reached its planned end when the next one began, so a summary of it can contain an outcome that nobody knew yet.

We evaluate logistic regression (LR), random forests (RF) and gradient boosting (GB) under time-ordered splits, leave-one-project-out (LOPO) testing and ten external projects. Each model is compared with the simplest rule that could do the same job. Two plausible claims fail that comparison: that a learned model warns of spillover better than counting open issues, and that excluding unfinished sprints from history features improves accuracy. Table II links each headline result to its model, split, population and comparator.

TABLE I
THE NEAREST PRIOR WORK.

Study	Unit	Target	Prediction-time state
[2]–[4]	Issue	Story points	Not applicable
[5]	Sprint	Delivery and quality	Not reported
[6]	Iteration	Velocity	Not reported
[7]	Issue	Delay	At prediction time
[8]	Iteration	Delivered vs. committed	Rebuilt (change log)
[9]	Sprint	Performance class	End-of-sprint burndown
[10]	Sprint	Deliverability	Not reported
This work	Sprint	Spillover, late closure	Rebuilt, fallbacks logged

II. Related Work

Machine learning on Agile data has mostly targeted effort. Story points have been estimated from issue text [2] and issue fields [3], and a TAWOS replication [4] found that the deep model did not consistently beat mean and median baselines. Sprint2Vec [5] regresses delivery on learned sprint embeddings, and Hearty et al. [6] forecast velocity with a dynamic Bayesian network. Choetkiertikul et al. predicted issue delays from networked issue data [7].

The closest study is by Choetkiertikul et al. [8], who predicted the gap between committed and delivered story points for 3,834 iterations of five Jira projects, MongoDB among them. They rebuilt each iteration's state at the prediction time from the issue change logs, at the start and at 30%, 50% and 80% of the iteration. Our reconstruction follows the same principle, and we do not claim it as new. Our additions are a record of where the change log runs out, a timing rule for history features when sprints overlap, and an analysis of what each of two binary labels measures. We also add learning-free count and rate baselines, within-project scoring and reproduced leakage mechanisms. Table I compares these studies. Our literature search was not systematic.

Two sprint-level studies illustrate the leakage mechanisms we test. Pérez Castillo et al. [9] classified sprint performance from eleven burndown features, one of which is the share of work completed by the end of the sprint, so the inputs

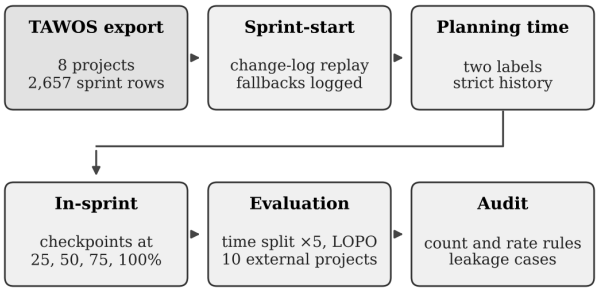


Fig. 1. Pipeline. The TAWOS stages share one cohort and one per-project time split; the repeats and external projects are described in Sections III-D and IV-D.

contain the outcome. Obike et al. [10] reported an AUC of 0.9995 for deliverability after SMOTE oversampling [11] and do not describe a train-test split made before it. We did not inspect either implementation. Instead we reproduce both mechanisms on our own data (Section IV-H), following the leakage taxonomy of Kapoor and Narayanan [12]. Falessi et al. [13] showed for defect prediction that validation must preserve time order.

We compute Shapley values [14] exactly over feature groups in the interventional form [15], because correlated metrics distort per-feature readings [16]. Our in-sprint setting is a case of outcome-oriented predictive process monitoring [17], [18], in which a running case is scored at successive prefixes.

III. Data and Methods

Fig. 1 outlines the pipeline.

A. Dataset and Sprint-Start Reconstruction

We use the Sprint, Issue and Change_Log tables of TAWOS [1] for the eight projects with public Jira data and at least 30 sprint records. They are Apache Mesos, Appcelerator Studio, Atlassian Confluence Cloud and Server, LSST Data Management, Mule, Titanium SDK and Titanium Mobile. The export holds 2,657 sprint rows, 145,309 issues and 88,364 change events on the Sprint and Story Points fields, whose names we match without regard to case. We exclude 44 sprints still marked FUTURE or ACTIVE and three closed sprints that end before they start. Because history features need a predecessor, we also drop each project's first eligible sprint, which leaves 2,602 modelled sprints. Sprint lengths range from under a day to over 60 days, and we keep them all.

Our procedure starts from the latest recorded state and replays each issue's Sprint and Story Points changes back to the sprint-start timestamp. An issue counts as committed only if, at that moment, it existed, belonged to the sprint and was unresolved. This yields 20,603 issue-sprint commitments, 20,579 of them in modelled sprints. Where an issue has no recorded change for a field, we use its static value and log the fallback. Of all commitments, 7.2% rely on the static sprint link and 45.1% on static story points. Moreover, 73.5% of issues have no story points, which we code as zero; dropping the point features leaves the late-closure AUC unchanged at 0.751. Coverage of observed sprint history ranges from 54.1% of commitments in Confluence Cloud to 99.1% in Confluence Server, so Section IV-B repeats the analysis on provenance-restricted subsets.

At the start instant, 911 sprints (35.0%) hold no committed issue and 1,691 hold at least one. Many were filled later: in 33% the first issue was added within an hour of the start, and in 42% within a day. We keep the exact start instant as the boundary, because anything later is not planning-time scope.

B. Outcomes

Late closure is 1 when the completion timestamp is more than a day after the planned end. It is defined for all 2,602 sprints, including empty ones, and Section IV-A also reports it on non-empty sprints and at longer thresholds. Spillover is 1 when at least one committed issue is unresolved at the planned end. We score it only on the 1,691 non-empty sprints, because an empty sprint cannot spill. The label is 93% positive overall and 94% in the test set, whose 29 negatives include 10 from Mule. Unfinished issues are carried forward. Of the 20,579 commitments, 32% are resolved by the planned end, 35% more than 30 days after it and 12% never, and 43.5% re-commit an issue already committed to an earlier sprint. Spillover labels are therefore not independent across sprints, and scoring at the actual closure instead changes the spillover rate by only one point. Three numeric targets accompany the labels: issues completed and carry share (the fraction of committed issues unfinished) on non-empty sprints, and days late, floored at 0 and capped at 30, on all sprints.

C. Features and the Parallel-Sprint Timing Rule

All features are read from the reconstructed sprint-start state. TAWOS has no team or board identifier, so history features are computed per project. Scope is the committed issue count and story points, and sprint length is the planned duration. Recent history summarises the project's earlier sprints (mean delivered points and mean completion ratio over the last three, the last carry-over, and the number of such sprints), and project identity is one-hot encoded. The legacy set summarises the immediately preceding sprints whether or not they had finished. The strict set differs in two ways. First, it admits only sprints that completed before the current start. Second, it adds the number of other sprints open at the start (mean 1.9, maximum 11), capacity and past lateness. Capacity is the mean number of issues completed and committed over recent sprints, plus committed issues divided by mean issues completed. Past lateness is the mean days late and the share of late sprints. Section IV-B separates the two changes. Results use the strict set unless marked legacy.

D. Models and Validation

The learning-free baselines are an always-positive rule, the project's training late rate, the project's late rate over its last three finished sprints, and the committed issue count. The scope-only LR uses committed issues, story points and sprint length. The learned models are LR on all features, an RF

TABLE II

HEADLINE RESULTS. n (POS.): TEST SPRINTS (POSITIVES); CHECKPOINT ROWS COUNT SPRINTS STILL OPEN. DIFFERENCES ARE ROW MINUS COMPARATOR WITH 95% PROJECT-CLUSTER BOOTSTRAP INTERVALS. BRACKETS IN THE RESULT COLUMN ARE AUC INTERVALS (ROW 2: WITHIN-PROJECT SPRINT BOOTSTRAP; ROW 8: PROJECT-CLUSTER BOOTSTRAP). ROWS 4 AND 7 GIVE MEAN $\pm$ SD OVER FIVE TIME-ORDERED REPEATS AND OVER THE EIGHT EXTERNAL PROJECTS WHERE LATE CLOSURE IS MEASURABLE. ROWS 9 TO 11 AND 14 USE NO MODEL; ROW 14 IS PAIRED WITH A REBUILT COPY OF THE ROW 13 MODEL (AUC .958). SP: STORY POINTS.

	Task	Split	n (pos.)	Model, features	Result	Difference [95% CI]	Comparator
1	Late >1 day	Time	786 (246)	GB, strict	AUC .751	+.111 [.028, .258]	Project rate .640
2	Late >1 day	Time	786 (246)	GB, strict	Within-project .634 [.570, .695]	—	Past late rate .510 within
3	Late >7 days	Time	786 (87)	GB, strict	AUC .638; within .502	—	Project rate .623
4	Late >1 day	Time ×5	516–524 (139–240)	GB, strict	AUC .696 $\pm$ .054	—	Shuffled 70/30 .765 $\pm$ .017
5	Late >1 day	LOPO	2,602 (1,006)	RF, strict	AUC .642	+.110 [.018, .215]	Scope-only LR .532
6	Late >1 day	To MariaDB	391 (106)	GB, strict, no SP	AUC .710; per project .699, .592	—	Scope-only LR .576
7	Late >1 day	8 of 10 external	21–269 each	GB, strict, no SP	Mean AUC .620 $\pm$ .090	CI above .5 in 2 of 8	Scope-only LR .362
8	Spillover	Time	511 (482)	LR, scope only	AUC .885 [.818, .922]	+.045 [−.010, .088]	RF, strict .840
9	Spillover	Time	511 (482)	Committed issue count	AUC .881	—	Row 8
10	Carry share $\geq$ .25	Time	511 (474)	Committed issue count	AUC .775	—	Best model (RF) .730
11	Carry share $\geq$ .5	Time	511 (440)	Committed issue count	AUC .562	—	Best model (RF) .615
12	Spillover, 25%	Time	507 (478)	GB, strict + progress	AUC .926	+.097 [.054, .163]	No progress .830
13	Spillover, 50%	Time	501 (472)	GB, strict + progress	AUC .957	+.123 [.068, .188]	No progress .835
14	Spillover, 50%	Time	501 (472)	Open committed count	AUC .945	−.013 [−.034, .010]	Row 13

[19] with 300 trees, depth 5 and minimum leaf 3, and scikit-learn [20] histogram GB with 200 iterations, depth 3 and learning rate 0.05. LR and RF use balanced class weights and GB does not, so their F1 and Brier scores are not directly comparable. We therefore rank models by the area under the receiver operating characteristic curve (ROC-AUC). Numeric targets use ridge regression, RF and GB with squared or absolute loss. Their baselines are the training median, the prior-sprint value, the project median and committed issues times the recent completion rate. We ran no hyperparameter search. On TAWOS the classification threshold is fixed at 0.5, and for the repeats and external projects it is set by Youden's J on out-of-fold training predictions.

The main split sorts each project's sprints by time and trains on the earliest 70%. This gives 1,816 training and 786 test sprints for late closure, and 1,180 and 511 for spillover. In 29 training sprints (1.6%) the label became known only after the project's first test sprint had started. Because pooled AUC also rewards separating projects, we report the mean within-project AUC for late closure. We repeat the time split five times, training on the first 40% to 80% of each project and testing on the next 20%, and contrast this with five shuffled 70/30 splits [21], [13]. LOPO trains on seven projects and tests on the eighth, whose history features still come from its own past (Section IV-C). Intervals resample the eight projects 2,000 times. With eight clusters they are approximate and uncorrected for multiple comparisons, so we treat them as exploratory. GB is deterministic under these settings, and the RF late-closure AUC has an SD of 0.003 over ten seeds.

E. Explanations and Checkpoints

We compute interventional Shapley values exactly over feature groups. Planning-time models use seven groups (128 coalitions): scope, sprint length, parallel sprints, recent history, capacity, past lateness and project. The capacity group includes committed issues divided by capacity, so it partly carries scope. Checkpoint models use nine groups (512 coalitions), in which capacity joins recent history and progress, time pressure and scope change are added. A coalition's features are fixed to the sprint's values, and the others are drawn from 40 training sprints. We report each group's share of the mean absolute attribution over 150 test sprints (100 at checkpoints).

TABLE III

PLANNING-TIME RESULTS, STRICT FEATURES, TIME-ORDERED SPLIT, THRESHOLD 0.5. WITHIN: MEAN AUC COMPUTED SEPARATELY IN EACH OF THE EIGHT PROJECTS, FROM A SECOND RUN OF THE SAME CODE (POOLED RF AUC .710 IN THAT RUN).

Task	Model	n	F1	AUC	Within	Brier
Late	Always positive	786	.477	.500	—	—
Late	Scope-only LR	786	.336	.530	.473	.244
Late	Project rate	786	.470	.640	—	.221
Late	Past late rate	786	—	.635	.510	—
Late	LR	786	.495	.680	.492	.224
Late	RF	786	.502	.715	.603	.207
Late	GB	786	.553	.751	.634	.176
Spill.	Always positive	511	.971	.500	—	—
Spill.	Committed count	511	—	.881	—	—
Spill.	Scope-only LR	511	.696	.885	—	.230
Spill.	LR	511	.842	.827	—	.161
Spill.	RF	511	.880	.840	—	.115
Spill.	GB	511	.974	.812	—	.049

For the in-sprint analysis we rebuild the state at 25%, 50%, 75% and 100% of the planned duration from events at or before each point. The progress features are committed issues done and remaining, points left, the gap between work done and time elapsed, the observed and required completion rates, and scope added or removed. A sprint already closed at a checkpoint leaves that checkpoint's population. Of all 2,602 sprints, 98.6% are still open at 25%, 96.2% at 75% and 65.5% at 100%. Spillover is not scored at 100%, because the state determines it. Each progress-aware model is compared with the same algorithm without progress features and with two rules: the count of committed issues still open, and one minus the share done.

IV. RESULTS

A. Planning-Time Classification

Table III gives the main time-split results. For late closure, GB reaches a pooled AUC of 0.751, RF 0.715 and LR 0.680, while the scope-only model scores 0.530. Only GB's paired

interval excludes the project late rate of 0.640 (Table II, row 1). Much of the pooled AUC comes from separating projects. Within a project GB averages 0.634 (interval 0.570 to 0.695), RF 0.603 and LR 0.492 (row 2). Per-project values range from 0.409 to 0.828, on test sets with as few as six late sprints. The project's late rate over its last three finished sprints scores 0.635 pooled and 0.510 within a project, so it identifies late projects but not late sprints. The 70% cut is also the most favourable one. Over five time-ordered repeats GB averages 0.696 ± 0.054 (row 4), from 0.62 with 40% of each history for training to 0.75 with 70%. On non-empty sprints only, GB scores 0.713 pooled and 0.586 within a project.

The signal is concentrated in short lags. Among late sprints the median delay is 4.0 days, and 40% close one to three days late. Sprints whose planned end falls on a Friday or a weekend (38% of all sprints) are late 57% of the time, against 28% for the rest. With a three-day threshold GB scores 0.727 pooled against 0.705 for the project rate, and 0.623 within a project. With a seven-day threshold it scores 0.638 against 0.623, and 0.502 within a project (row 3). Forgiving only closures by the end of the next working day lowers the late rate from 38.7% to 25.4%. GB then scores 0.727 pooled and 0.631 within a project (interval 0.531 to 0.689), against 0.482 for the past late rate, but on the external projects (Section IV-D) its mean AUC falls from 0.620 to 0.546, with two of nine intervals above 0.5. Within a project, smaller committed scope also goes with later closure (count AUC 0.422). Late closure therefore mostly records how promptly a board is closed, including idle boards, rather than whether work overran.

Spillover looks easier, but the result means less than it seems. The scope-only model scores 0.885 (interval 0.818 to 0.922), and every richer model scores lower. Its advantage over RF, the strongest of them, is 0.045 with an interval that includes zero (row 8). The number of committed issues alone scores 0.881 (row 9). A label that asks whether any of n issues is unfinished favours large n, and with 29 test negatives a few sprints move the AUC. The 0.885 shows that the label is easy to rank, not that sprint risk is predictable; Section IV-F tests size-normalised labels. Keeping the 275 empty test sprints would inflate AUC by about 0.13 without adding skill: the legacy RF scores 0.986 on all 786 against 0.852 on the 511 non-empty ones, and the non-empty flag alone scores 0.956.

B. Timing, Provenance and Label Defects

Moving from legacy to strict history features raises late-closure AUC under LOPO for LR (0.542 to 0.629) and RF (0.600 to 0.642), but not for GB (0.632 to 0.627). The rise comes from the added past-lateness features. Applying the timing rule alone to the legacy features lowers LOPO AUC by 0.023 for LR (interval -0.074 to 0.023) and 0.039 for RF (-0.053 to -0.007). Adding only the two past-lateness features to the legacy set raises it by 0.103 for LR and 0.081 for RF, both with intervals above zero. Removing them from the strict set drops LR to 0.496 and RF to 0.563. Under LOPO, past lateness therefore stands in for project identity. Legacy history plus past lateness reaches 0.682 for RF, above the strict set, so the unrestricted history still carries cross-project signal; we exclude it because it was not available at planning time. On the time split the timing rule changes little (LR 0.636 to 0.661). Spillover barely changes under either set, because scope is the same in both.

Provenance limits the spillover result. We rerun the scope-only model and the legacy RF on two nested subsets. When every committed item must have a Sprint-field change recorded before the start, 1,174 sprints remain (356 in the test period, 335 positive), and the models score 0.875 and 0.850. When a recorded Story Points change is also required, 358 remain (110 in the test period, 94 positive), and they score 0.772 and 0.652. The Hanley–McNeil [22] standard error of 0.054 gives an approximate 95% range of 0.67 to 0.88. This subset is also a different population, 85% positive against 94%, so 0.772 is not a corrected estimate. It shows only that the 0.885 is not established on fallback-free data.

Completion is read from each issue's resolution date, and in Confluence Cloud 51% of commitments have none (3% to 16% elsewhere). At the planned end, 74% of its non-empty sprints show no committed issue done, against 48% in Confluence Server and 11% to 31% in the other projects. Its spillover labels and progress features are therefore unreliable. Without Confluence Cloud the scope-only spillover AUC rises from 0.885 to 0.908. Late closure uses the sprint's own completion stamp and is unaffected.

C. Cross-Project Generalisation

Because LOPO keeps each held-out project's own earlier sprints as history inputs, it asks whether a model learned on seven projects carries over to an eighth that has a track record. It does not test a cold start. Strict-feature RF reaches 0.642 against 0.532 for scope only (row 5), and GB 0.627. With legacy features, the late-closure RF pools to 0.600 but ranges from 0.531 on Titanium Mobile to 0.780 on Appcelerator Studio. The non-empty spillover RF ranges from 0.706 on Confluence Cloud to 0.950 on Appcelerator Studio. These project ranges of about 0.25 dwarf the spread between models (0.015 under LOPO). Under LOPO the always-late rule (F1 0.558) has a higher F1 than every learned model, because a 0.5 cut-off on probabilities learned elsewhere does not transfer.

D. External Projects

We ran the frozen pipeline on two MariaDB projects, MaxScale and ColumnStore (391 sprints from 2016 to 2026, 106 late). They were converted to the TAWOS tables without changing reconstruction, labels, features or models. MariaDB records no story points, so these runs drop the point features in training and testing. Trained on TAWOS alone without project identity, GB ranks the pooled MariaDB late closures at 0.710 (sprint-bootstrap interval 0.637 to 0.773). Per project it scores 0.699 on ColumnStore (54% late) and 0.592 on MaxScale (15% late), so the pooled value again benefits from separating projects (row 6). On MariaDB's own time split, GB (0.690) scores below the project late rate (0.745; difference -0.055, interval -0.125 to 0.019).

TABLE IV
IN-SPRINT ROC-AUC BY CHECKPOINT FOR GB WITH AND WITHOUT PROGRESS FEATURES AND FOR TWO LEARNING-FREE RULES. EACH CHECKPOINT SCORES ONLY THE SPRINTS STILL OPEN. UNDER LOPO THE RULES ARE COMPUTED ONCE OVER ALL OPEN SPRINTS.

Task	Model	Split	25%	50%	75%	100%
Spill.	Planning only	Time	.830	.835	.827	—
Spill.	With progress	Time	.926	.957	.953	—
Spill.	Rule: open committed count	Time	.925	.945	.954	—
Spill.	Rule: 1 − share done	Time	.667	.767	.802	—
Spill.	Planning only	LOPO	.801	.815	.804	—
Spill.	With progress	LOPO	.860	.897	.927	—
Spill.	Rule: open committed count	LOPO	.876	.902	.938	—
Spill.	Rule: 1 − share done	LOPO	.610	.715	.827	—
Late	Planning only	Time	.751	.751	.751	.717
Late	With progress	Time	.754	.760	.749	.704
Late	Rule: open committed count	Time	.479	.483	.486	.484
Late	Planning only	LOPO	.645	.652	.647	.672
Late	With progress	LOPO	.658	.667	.664	.657
Late	Rule: open committed count	LOPO	.518	.517	.521	.510

We then added eight MongoDB projects from MongoDB's public Jira (Compass, WiredTiger, Shell, Database Tools and the Java, C, C++ and PHP drivers; 526 sprints). All ten external projects were scored one at a time with the threshold set on TAWOS. Late closure is measurable in eight, since the other two have only three on-time sprints each. Mean AUC is 0.620 ± 0.090 (row 7). The point estimate exceeds 0.5 in seven projects, but with 21 to 269 sprints each only two intervals exclude 0.5 (ColumnStore and the PHP driver). The scope-only model scores below 0.5 in seven of eight projects (mean 0.362), so on these projects, as within TAWOS projects, larger committed scope goes with on-time closure. Spillover is measurable in only four projects, each with 6 to 9 negatives (mean AUC 0.734, scope only 0.742).

E. Explanations

We report planning-time Shapley shares for the two RF models only; GB, the stronger late-closure model, is not covered. For late closure, past lateness takes 38% of the attribution and project identity 22%, followed by sprint length (13%), capacity (12%) and committed scope (9%). For spillover, committed scope takes 41% and capacity 37%, and capacity partly restates scope. A group ablation on the legacy RF agrees. Removing scope lowers non-empty spillover AUC from 0.852 to 0.652 and late closure only from 0.705 to 0.690.

F. Carry Share and Other Numeric Targets

Carry share is the size-normalised form of spillover, and at planning time it is almost unpredictable. On the time split, R^2 is at most 0.02 for every model, and GB with absolute-error loss lowers the mean absolute error only from 0.218 for the prior-sprint value, the strongest simple baseline, to 0.201 (difference -0.017, interval -0.029 to -0.003). When a sprint is labelled troubled once a quarter of its committed issues are unfinished (93% of test sprints), the committed count scores 0.775 and the best model, RF, 0.730. At a half (86%) the count scores 0.562 and RF 0.615 (rows 10 and 11), so the model leads only where the count is weak. Leaving out Confluence Cloud, whose resolution dates are unreliable, does not rescue the half threshold (78% positive): GB scores 0.565 against 0.621 for the past carry share, and within a project the committed count (0.621) beats GB (0.521). At mid-sprint the share still open scores 0.810 and GB 0.839, a gap whose interval includes zero. Issues completed is close to arithmetic: committed issues times the recent completion rate gives a mean absolute error of 2.10 against 1.96 for GB, with an interval that includes zero. For days late, GB's gain over the training median also has an interval that includes zero.

G. In-Sprint Early Warning

Table IV shows the effect of progress features. For spillover on the time split, GB rises from 0.830 on planning features to 0.926 at 25%, 0.957 at 50% and 0.953 at 75% (rows 12 and 13). The count of committed issues still open, with no model, scores 0.925, 0.945 and 0.954. Under LOPO it scores higher than GB at every checkpoint (0.876, 0.902 and 0.938 against 0.860, 0.897 and 0.927). A rebuilt copy of GB, within 0.003 AUC of the original, exceeds the count on the time split by at most 0.013, and every paired interval includes zero (row 14). The share-based rule scores far lower (0.667 to 0.802) because it ignores how many issues remain.

For late closure, GB scores 0.754, 0.760, 0.749 and 0.704 with progress, against 0.751, 0.751, 0.751 and 0.717 without. Of 24 paired comparisons with the same model without progress, 22 intervals include zero, and the other two gains are below 0.03. At 50%, the in-sprint groups (progress, time pressure and scope change) take 61% of the spillover attribution and 18% of the late-closure one, where past lateness still holds 37%.

H. Leakage Case Studies

We reproduced two mistakes with a legacy-feature LR and scored F1 at the fixed threshold. Adding the sprint's own completion ratio, known only after the sprint, raises spillover F1 from 0.756 to 0.991 and AUC from 0.834 to 1.000, while late-closure F1 rises by only 0.010. For oversampling we compare random duplication of the minority class in the training part only with duplication over the whole pool before splitting. For late closure the second raises F1 from 0.457 to 0.646 and AUC from 0.638 to 0.688. Spillover is unchanged, since at 94% positives there is little to oversample. Shuffling sprints before splitting gives late closure 0.765 ± 0.017 against 0.696

$\pm$ 0.054 in time order (row 4), but the late rate of the time-ordered test windows ranges from 27% to 47%, so part of this gap may be drift rather than leakage.

V. Threats to Validity

Construct. The completion timestamp records when a sprint was closed, and its lateness depends partly on the weekday of the planned end and on idle boards. Spillover mostly records rollover, because most commitments are resolved weeks after the planned end and are committed again. History is per project, and a project can run several boards at once.

Reconstruction. Unrecorded changes cannot be recovered, and the story-point fallback limits the spillover estimate (Section IV-B). We do not reconstruct edits to the sprint's planned dates or issue status, so sprint length and both labels may carry later edits. Completion uses the final resolution date, which is often missing in Confluence Cloud and counts a reopened issue at its later resolution. Dates are in UTC, so the working-day rule can misplace a closure by a day.

Evaluation. Eight projects give wide, uncorrected cluster intervals, and several within-project test sets have fewer than ten late sprints. The within-project, threshold, count-rule and timing analyses come from a second run of the same code under scikit-learn 1.9. It reproduces the late-closure GB exactly and other models within 0.007 AUC. We ran no decision-utility analysis, and the Shapley values describe the fitted forests and say nothing about causes.

External. All projects use public Jira trackers. The external checks add two sources and ten projects, with spillover measurable in only four.

VI. Conclusion

Rebuilding sprint state from the change log is not new [8]; our contribution is an audit of what such data can support. Late closure ranks at about 0.70 AUC over repeated time splits, but most of that separates projects and short closing lags: within a project GB averages 0.634, the project's own late rate is at chance, the skill disappears at a seven-day threshold, and transfer to external projects is weak. Spillover as a yes-or-no label is close to mechanical, since the committed count at planning (0.881) and the open count at mid-sprint (0.945) match our best models. Excluding unfinished sprints from history costs 0.02 to 0.04 AUC across projects, the price of removing information nobody had. We recommend that such studies (1) rebuild the state at the prediction time, (2) report within-project as well as pooled results, (3) test label thresholds, (4) drop empty sprints for spillover, (5) keep time order and test leakage separately for each label, and (6) compare every model with a count or rate rule. None of our models is ready for deployment. Future work should reconstruct sprint dates and issue status, use board-level history, and look for planning-time signals of how much of a sprint slips.

All results use seed 42, Python 3.13 and pandas 2.2, with scikit-learn 1.8 for the main runs and 1.9 for the rest. The artefact at https://github.com/hani-ziad/SwEng_202610 holds the TAWOS export (Apache 2.0), the code, the scripts that pull the MariaDB and MongoDB data from their public Jira, 50 automated tests and every result file behind these numbers.

References

[1] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, "A versatile dataset of agile open source software projects," in Proc. 19th Int. Conf. Mining Softw. Repositories (MSR), 2022, pp. 707–711.

[2] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for estimating story points," IEEE Trans. Softw. Eng., vol. 45, no. 7, pp. 637–656, 2019.

[3] S. Porru, A. Murgia, S. Demeyer, M. Marchesi, and R. Tonelli, "Estimating story points from issue reports," in Proc. 12th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE), 2016, pp. 1–10.

[4] V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," IEEE Trans. Softw. Eng., vol. 49, no. 4, pp. 2677–2697, 2023.

[5] M. Choetkiertikul, P. Banyongrakkul, C. Ragkhitwetsagul, S. Tuarob, H. K. Dam, and T. Sunetnanta, "Sprint2Vec: A deep characterization of sprints in iterative software development," IEEE Trans. Softw. Eng., vol. 51, no. 1, pp. 220–242, 2025.

[6] P. Hearty, N. Fenton, D. Marquez, and M. Neil, "Predicting project velocity in XP using a learning dynamic Bayesian network model," IEEE Trans. Softw. Eng., vol. 35, no. 1, pp. 124–137, 2009.

[7] M. Choetkiertikul, H. K. Dam, T. Tran, and A. Ghose, "Predicting delays in software projects using networked classification," in Proc. 30th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), 2015, pp. 353–364.

[8] M. Choetkiertikul, H. K. Dam, T. Tran, A. Ghose, and J. Grundy, "Predicting delivery capability in iterative software development," IEEE Trans. Softw. Eng., vol. 44, no. 6, pp. 551–573, 2018.

[9] Y. J. Pérez Castillo, S. D. Orantes Jiménez, and P. O. Letelier Torres, "Sprint management in Agile approach: Progress and velocity evaluation applying machine learning," Information, vol. 15, no. 11, Art. no. 726, 2024.

[10] P. G. Obike, V. E. Ekong, and O. U. Obot, "A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning," Int. J. Comput. Appl., vol. 187, no. 49, pp. 40–53, 2025.

[11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[12] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, Art. no. 100804, 2023.

[13] D. Falessi, J. Huang, L. Narayana, J. F. Thai, and B. Turhan, "On the need of preserving order of data when validating within-project defect classifiers," Empir. Softw. Eng., vol. 25, pp. 4805–4830, 2020.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst. 30, 2017, pp. 4765–4774.

[15] D. Janzing, L. Minorics, and P. Blöbaum, "Feature relevance quantification in explainable AI: A causal problem," in Proc. 23rd Int. Conf. Artif. Intell. Statist. (AISTATS), PMLR vol. 108, 2020, pp. 2907–2916.

[16] J. Jiarpakdee, C. Tantithamthavorn, and A. E. Hassan, "The impact of correlated metrics on the interpretation of defect models," IEEE Trans. Softw. Eng., vol. 47, no. 2, pp. 320–331, 2021.

[17] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, "Outcome-oriented predictive process monitoring: Review and benchmark," ACM Trans. Knowl. Discov. Data, vol. 13, no. 2, Art. no. 17, 2019.

[18] S. A. Fahrenkrog-Petersen, N. Tax, I. Teinemaa, M. Dumas, M. de Leoni, F. M. Maggi, and M. Weidlich, "Fire now, fire later: Alarm-based systems for prescriptive process monitoring," Knowl. Inf. Syst., vol. 64, no. 2, pp. 559–587, 2022.

[19] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[20] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[21] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," IEEE Trans. Softw. Eng., vol. 43, no. 1, pp. 1–18, 2017.

[22] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," Radiology, vol. 143, no. 1, pp. 29–36, 1982.